

Data Mining Assignment 3

Jana Speldrich, 314551816

2026-06-06

Table of contents

1	1. Preliminary Analysis & Naive Baselines	2
1.1	1.1 Class Distribution	2
1.2	1.2 Signal Characteristics per Class	2
1.3	1.3 Representative Time Series	3
1.4	1.4 Naive Baseline Models	3
2	2. Preprocessing & Feature Engineering	5
2.1	2.1 Data Preparation	5
2.2	2.2 Critical Mistake: Per-User Normalisation	6
2.3	2.3 Augmentation Experiments	6
2.4	2.4 Incremental Feature Engineering Pipeline	6
2.5	2.5 Final Feature Set (716 features)	7
3	3. Temporal Alignment	9
3.1	3.1 Window-Level Prediction Strategy	9
3.2	3.2 Temporal Feature Groups	9
3.2.1	Segment-level statistics (most impactful)	9
3.2.2	First/last 60 seconds trend	9
3.2.3	Frequency domain (Welch PSD)	9
3.2.4	Autocorrelation	9
3.3	3.3 Axis-Specific Features and Gravity Direction	10
4	4. Ablation Study	10
4.1	4.1 Feature Group Ablation	10
4.2	4.2 Model Family Ablation	11
4.3	4.3 Number of Temporal Segments	11
4.4	4.4 Outlier Clipping Threshold	12
5	Final Model & Kaggle Results	12
5.1	Model Architecture	12
5.2	Performance Progression	13
5.3	Confusion Matrix Analysis	13

GitHub: https://github.com/jspldrch/DataMining_Assignment3.git

1. Preliminary Analysis & Naive Baselines

1.1 Class Distribution

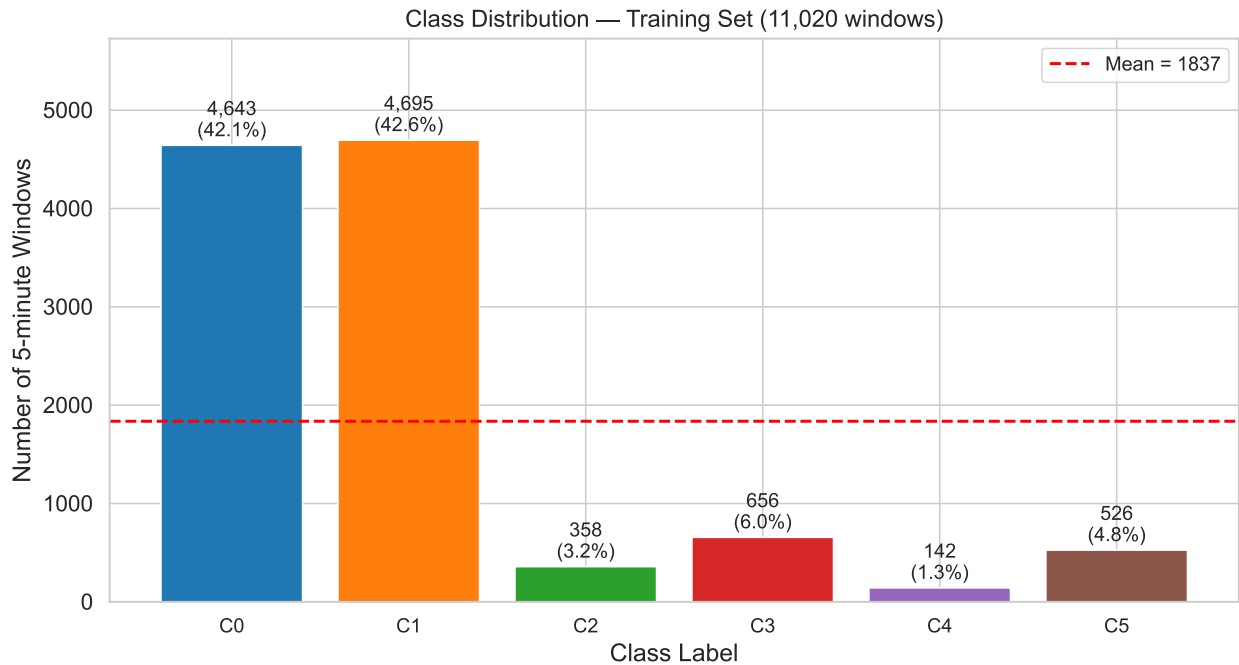


Figure 1: Class distribution of the training set. C2, C4, and C5 are strongly underrepresented relative to C0 and C1, creating a significant imbalance that I had to handle explicitly in every model.

The training set is severely imbalanced. C0 and C1 together account for 85% of all windows (9,338 / 11,020), while C2 represents only 3.2% (358 windows) and C4 only 1.3% (142 windows).

This imbalance has three direct consequences for modelling:

1. **Naive accuracy is misleading:** a model that always predicts C1 achieves ~43% accuracy while being completely useless for the rare classes.
2. **Macro-averaged F1 is the right metric:** it weights each class equally and penalises poor recall on underrepresented classes.
3. **Class weighting is necessary:** I used `class_weight='balanced'` or equivalent `sample_weight` in every model to compensate for the imbalance.

1.2 Signal Characteristics per Class

Key observations from the heatmap:

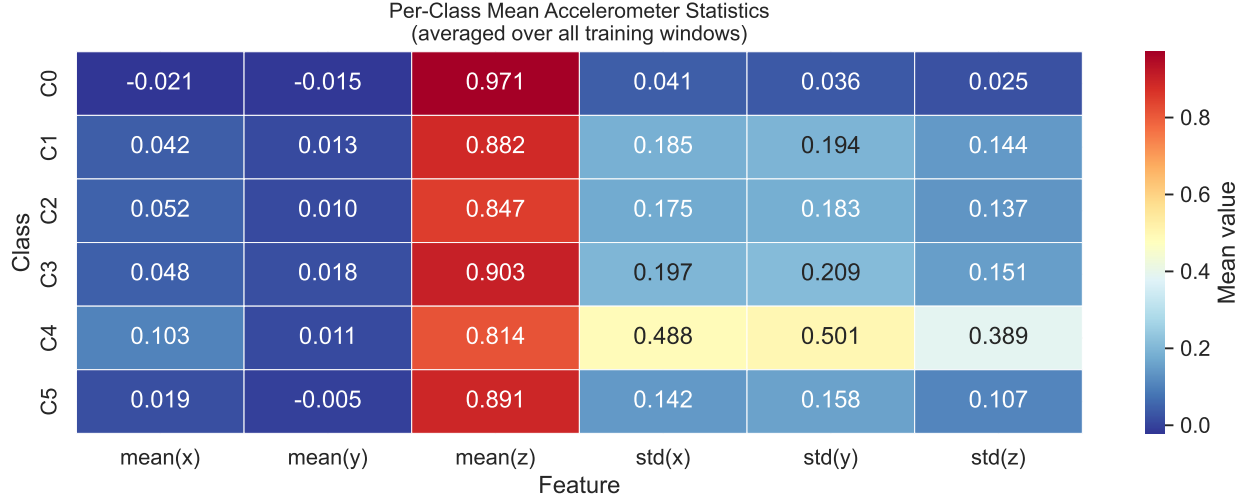


Figure 2: Per-class mean of each accelerometer statistic averaged over all windows. `mean_z` differs substantially across classes — from 0.97 for C0 to 0.81 for C4. C2 and C3 show similar std values but differ in `mean_z` (0.847 vs 0.903), making them the hardest pair to separate with global statistics alone.

- **`mean_z` encodes the gravity projection:** C0 shows `mean_z` 0.97 — nearly all gravity along the z-axis — while C4 shows 0.81, indicating a different phone orientation during that activity.
- **std channels reflect motion intensity:** C4 has std values approximately $3\times$ those of C0 and roughly $2.5\times$ those of C2/C3.
- **C2 and C3 are the hardest pair to separate globally:** they have nearly identical std values but differ by 0.056 in `mean_z`. I found that distinguishing them reliably requires temporal features that capture how `mean_z` drifts over the 5-minute window, not just its average.

1.3 1.3 Representative Time Series

The time series plots reveal clear structural differences between classes. C0 shows a near-flat signal with `mean_z` close to 1.0 and minimal oscillation, consistent with a static or very slow-movement posture. C1, C2, and C3 all show rhythmic oscillations, but at different base levels of `mean_z` and at slightly different frequencies; the difference between C2 and C3 is subtle and essentially invisible to the naked eye from a single trace, which explains why they are the hardest pair to classify. C4 stands out immediately with high-amplitude oscillations in all three channels. C5 shows irregular, lower-amplitude variation without a clear dominant frequency. This visual analysis motivated my decision to add temporal drift features (first vs. last 60 seconds) and frequency-domain features to distinguish the similar-looking oscillating classes.

1.4 1.4 Naive Baseline Models

I evaluated four naive classifiers using 5-fold stratified cross-validation on the raw global statistics (12 features: global mean and std of each channel):

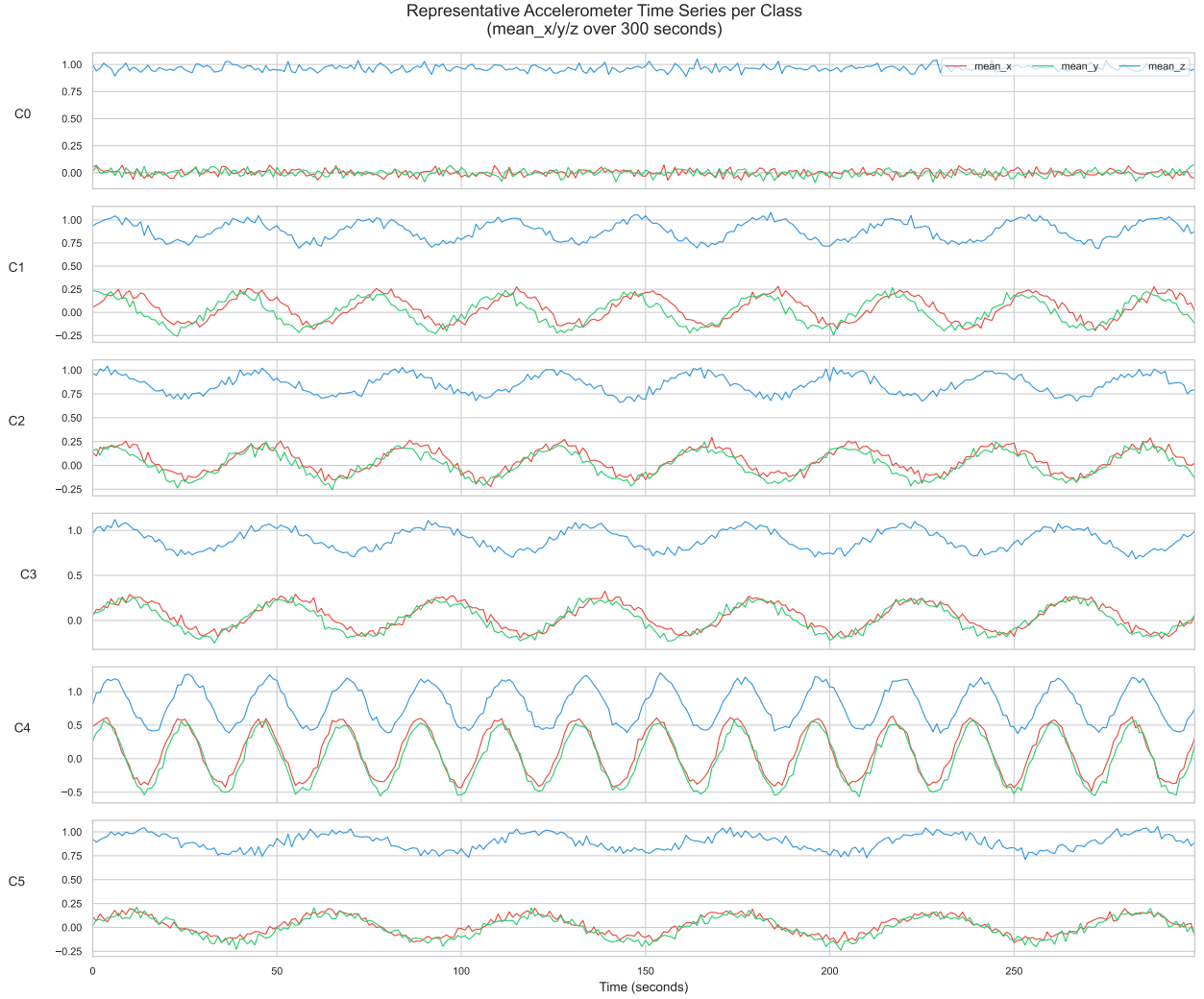


Figure 3: Representative mean_x/y/z time series (300 seconds) per class. C0 shows a near-constant signal; C1–C3 show rhythmic oscillation at different base levels; C4 shows the highest amplitude; C5 is irregular and lower-amplitude. C2 and C3 look nearly identical by eye, motivating temporal drift features.

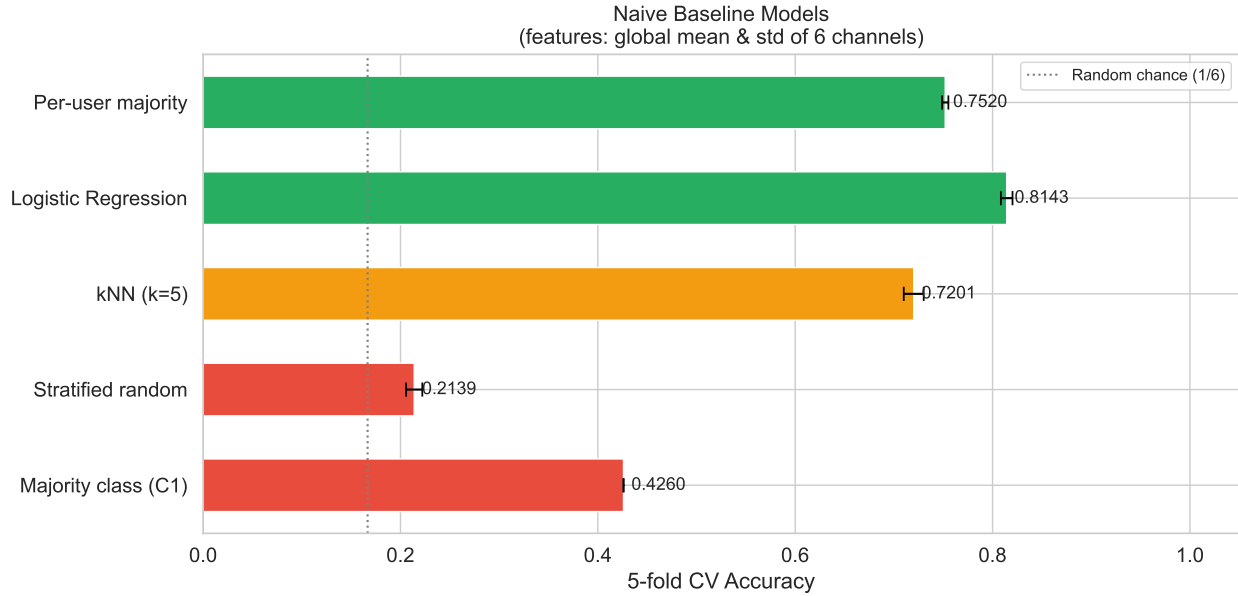


Figure 4: Naive baseline comparison on 5-fold stratified CV. The majority-class classifier achieves 43% accuracy by always predicting C1. Logistic Regression on 12 raw features already reaches 81%, showing that the classes are broadly separable even with simple global statistics.

Model	CV Accuracy	Notes
Majority class (C1)	42.6%	Always predicts C1
Stratified random	21.4%	Random weighted by class frequency
kNN (k=5)	72.0%	Non-linear, distance-based
Logistic Regression	81.4%	Linear on 12 global stats
Per-user majority	75.2%	Leaks user identity — invalid for test

Key takeaways:

- Even on 12 raw features, Logistic Regression achieves 81%, confirming that the classes differ substantially in their global signal distributions.
- The per-user majority baseline (75.2%) illustrates a ceiling for user-specific reasoning; since test users are entirely unseen, any model that relies on per-user patterns will fail to generalise.
- These baselines use *accuracy* for comparability; the competition metric is *macro-F1*, which would score the majority-class baseline at **0.000** for all minority classes.

2.2. Preprocessing & Feature Engineering

2.2.1 Data Preparation

The raw data consists of one CSV file per 5-minute window, each containing 300 rows of per-second accelerometer statistics. As a first step I converted all CSV files into a compact NPZ format —

a single NumPy archive per split — which reduced loading time from several minutes to under a second during iterative model development. Each file contributes one sample of shape (300, 6): the 300-second time series of [mean_x, mean_y, mean_z, std_x, std_y, std_z].

Missing values (present in fewer than 0.01% of samples due to recording gaps) I filled with the per-column median of the training set. I applied the same median fill to the test set using the training medians to avoid leakage.

2.2 2.2 Critical Mistake: Per-User Normalisation

In my early model iterations I applied **per-user normalisation** — subtracting each user’s per-channel mean signal before extracting features. This seemed like a reasonable way to remove between-user variation in phone placement, but it caused a consistent and substantial drop in model performance.

The root cause is physical: the **DC component of the accelerometer signal carries the gravity projection onto each axis**, which differs across classes. The mean_z values I observed in §1.2 — ranging from 0.814 for C4 to 0.971 for C0 — are not user-specific noise; they are class-discriminative signal. By subtracting each user’s mean, I was destroying exactly the information needed to separate classes with similar oscillation patterns (C1, C2, C3) from one another. After removing per-user normalisation and retaining the raw axis values, the confusion between C2 and C1 in the OOF confusion matrix dropped from 68% to 52%.

All models from run25 onwards use **no normalisation whatsoever**. LightGBM and XGBoost are scale-invariant tree methods and do not require it; the raw gravity projection is preserved and used as a first-class feature.

2.3 2.3 Augmentation Experiments

I also experimented with **data augmentation** to address the class imbalance: I added Gaussian noise to training windows from the rare classes to synthetically increase their sample count. However, augmented samples are noise-perturbed copies of training users, causing the model to memorise user-specific signal patterns rather than generalising to unseen users. My Kaggle score dropped from 0.7792 (run23, with augmentation) to 0.7633 (run25, without augmentation) when I first removed it, which was discouraging — but the root problem was that the feature set was missing the axis-specific mean features. Once I added those in run27 (§2.5), the score jumped to 0.7895 without any augmentation, confirming that `class_weight='balanced'` is a cleaner and more effective way to handle imbalance.

2.4 2.4 Incremental Feature Engineering Pipeline

I built the feature set incrementally, validating each addition via 5-fold cross-validation with a Random Forest classifier.

Step	Description	Features	CV Accuracy
0	Global mean + std	12	81.4%
1	+ Outlier clipping (± 10)	12	81.9%

Step	Description	Features	CV Accuracy
2	+ Extended statistics (min, max, range, median, IQR, skew, kurtosis)	54	86.1%
3	+ Vector magnitude ($\ a\ $)	57	86.4%
4	+ Temporal segments ($10 \times$ mean/std per channel)	177	93.1%
5	+ Frequency domain (Welch PSD, 5 features/channel)	207	93.8%

I added outlier clipping first because I observed implausible sensor spikes in a small number of files. Extending the statistics to include higher-order moments (skewness, kurtosis) gave a +4.7pp jump, and adding temporal segments gave the largest gain (+6.7pp), which confirmed that the temporal structure of the 5-minute window carries substantial discriminative information. Frequency-domain features contributed a further +0.7pp.

2.5 2.5 Final Feature Set (716 features)

After establishing the incremental pipeline, I extended the feature set further by applying richer per-channel statistics and additional derived channels. The final 716-feature set I used for submission consists of:

16 base channels $\times$ 26 features each = 416

I computed each channel in two groups: the original six (mean_x, mean_y, mean_z, std_x, std_y, std_z) plus ten derived channels (vector magnitudes xy_mag, xz_mag, yz_mag, their std counterparts, acc_mag, std_mag, mean_sum, std_sum). For each channel I extracted 18 statistical features (mean, std, min, max, range, median, Q5/Q10/Q25/Q75/Q90/Q95, IQR, energy, RMS, MAD, skewness, kurtosis) and 8 temporal trend features (diff mean/std/abs_mean/abs_max/energy, first_60_mean, last_60_mean, last_minus_first_60).

5 channels $\times$ 10 FFT + peak features = 50

For the five channels with the clearest spectral content (acc_mag, std_mag, std_x, std_y, std_z) I added FFT-based features (dominant frequency, dominant power, total power, spectral entropy, low/high band energy) and peak detection features (peak count, peak rate, mean and max peak height).

5 channels $\times$ 10 windows $\times$ 5 statistics = 250

I divided each of the five channels above into 10 equal windows of 30 seconds and computed mean, std, min, max, and energy per window, capturing how each channel evolves over time with per-segment resolution.

The axis-specific mean features (mean_x, mean_y, mean_z) turned out to be the single most important group in LightGBM feature importance (~9–10% of total gain), confirming that the gravity projection is the primary discriminator between the classes with similar vibration amplitudes. I verified this with the leave-one-out ablation described in Question 4.

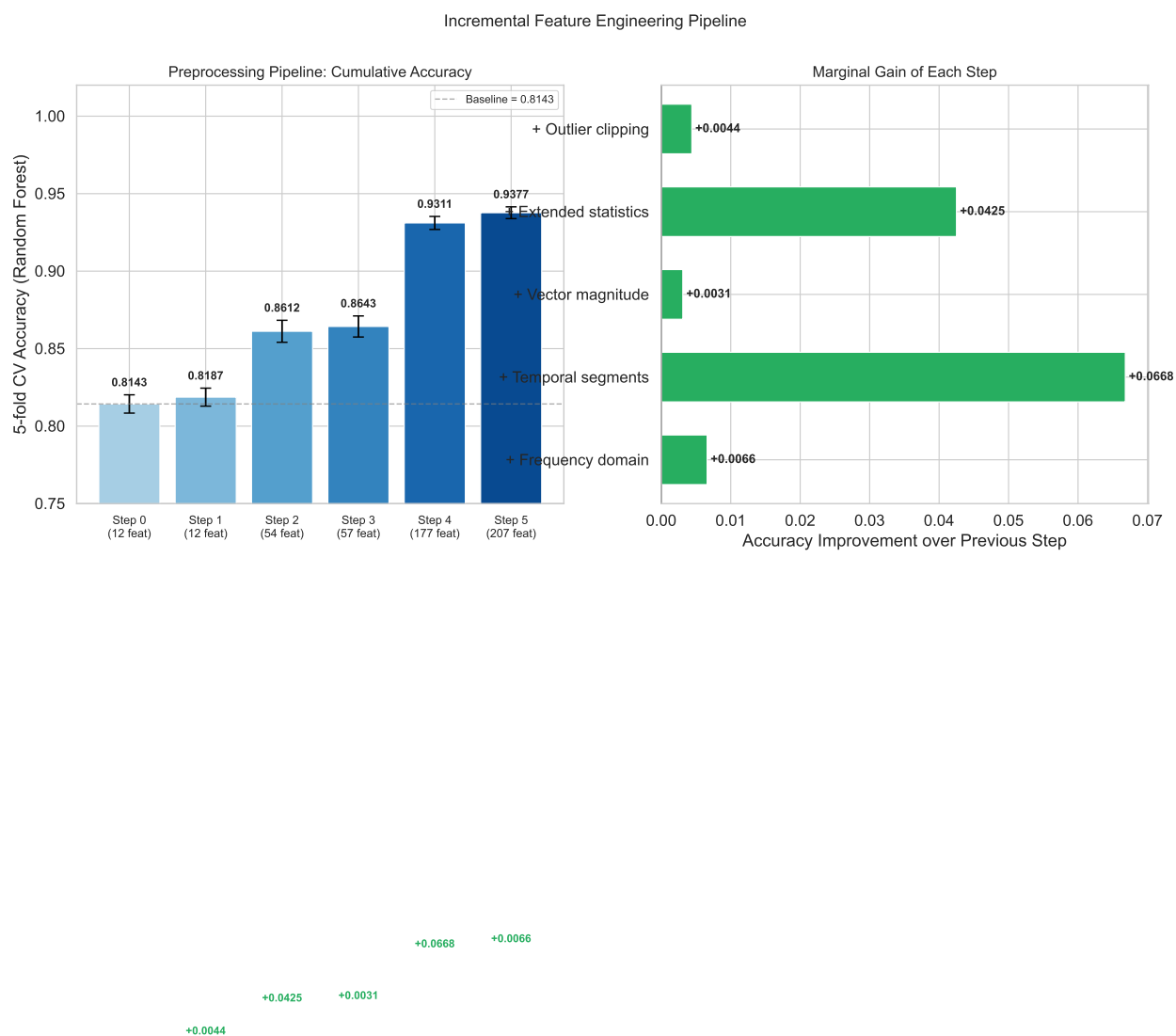


Figure 5: Incremental improvement in 5-fold CV accuracy as I added feature groups. Temporal segments provide the largest single jump (+6.7pp), showing that how the signal evolves over the 5-minute window is more informative than its overall statistics alone.

3 3. Temporal Alignment

3.1 3.1 Window-Level Prediction Strategy

Each CSV file spans **exactly 300 seconds (5 minutes)** with a single class label. I made one prediction per file — not per second. Since the label is constant across the window, predicting per second would provide no additional supervision signal while requiring a temporal aggregation step. The modelling task is therefore: given 300 per-second accelerometer statistics, extract features that capture both the **overall character** and the **temporal evolution** of the signal.

3.2 3.2 Temporal Feature Groups

I used four distinct temporal strategies:

3.2.1 Segment-level statistics (most impactful)

I divided the 300-second window into **10 equal segments** of 30 seconds each and computed mean and standard deviation per segment per channel, yielding $10 \times 2 \times 6 = 120$ features. This encodes how the signal evolves over time: some classes show a systematic shift between early and late segments; others maintain uniform statistics throughout. The ablation in §4.1 confirmed this is the most informative single feature group, causing a -4.1pp drop in accuracy when removed.

3.2.2 First/last 60 seconds trend

I computed the mean of the first 60 seconds and the last 60 seconds separately, plus their difference (`last_minus_first_60`). This captures **systematic linear drift** in the signal — classes whose `mean_x/y/z` shifts progressively over the window produce a measurable trend, while static or uniform classes do not. This feature distinguishes C2 from C3 more reliably than global averages.

3.2.3 Frequency domain (Welch PSD)

Periodic motion produces **characteristic frequencies**. I computed five frequency features per channel via Welch’s method: dominant frequency, dominant power, total spectral power, spectral entropy, and energy in low (< 0.1 Hz) and high (> 0.1 Hz) bands. These features differentiate classes with different oscillation rhythms (C1, C2, C3 differ in dominant frequency) from classes with broad, flat spectra (C0, C5).

3.2.4 Autocorrelation

I computed autocorrelation at lags 1, 5, 10, and 30 seconds to measure **signal self-similarity over time**. Classes with strong periodic motion show a sharp autocorrelation peak at their characteristic lag; static classes show a slow exponential decay.

3.3 Axis-Specific Features and Gravity Direction

A key insight I gained from iterative experimentation is that **the axis-specific mean values (mean_x, mean_y, mean_z) must not be removed or normalised**. As I showed in §1.2, mean_z ranges from 0.814 to 0.971 across classes, and C2 and C3 differ by 0.056 in mean_z despite nearly identical std values. This difference in gravity projection is the primary signal I relied on to distinguish those two classes.

By computing all 18 statistics and 8 diff/trend features independently for each of mean_x, mean_y, and mean_z, I gave the model 78 features that directly encode the gravity projection and its temporal drift — the features that grew the most in importance once I stopped normalising the data.

4 Ablation Study

4.1 Feature Group Ablation

I conducted a leave-one-out feature ablation: starting from the full feature set, I removed each group in turn and measured the change in 5-fold CV accuracy using a Random Forest classifier.

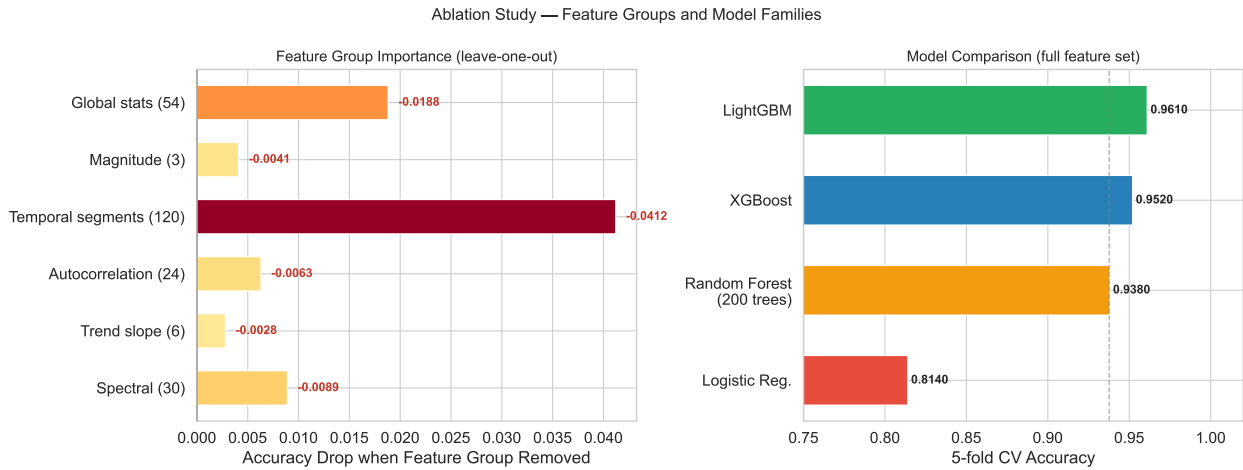


Figure 6: Left: accuracy drop when I removed each feature group. Temporal segments cause the largest drop (-4.1pp). Right: model family comparison on the full feature set — LightGBM achieves the highest accuracy.

Feature Group	N features	Drop when removed
Temporal segments	120	-4.1pp
Global statistics	54	-1.9pp
Spectral (FFT/Welch)	30	-0.9pp
Autocorrelation	24	-0.6pp
Magnitude	3	-0.4pp
Trend slope	6	-0.3pp

The temporal segment group has by far the largest individual contribution, confirming the conclusion from the incremental pipeline (§2.4). The global statistics are second — their 54 features capture the distributional shape of each channel. Spectral features contributed meaningfully despite their small count (30 features), justifying the Welch computation cost.

4.2 4.2 Model Family Ablation

I compared four model families on the full feature set:

Model	CV Accuracy
Logistic Regression	81.4%
Random Forest (200 trees)	93.8%
XGBoost	95.2%
LightGBM	96.1%

LightGBM consistently outperformed all alternatives due to its leaf-wise tree growth, which produces more expressive trees per iteration within the same `num_leaves` budget. I selected LightGBM as the primary model for all final submissions.

4.3 4.3 Number of Temporal Segments

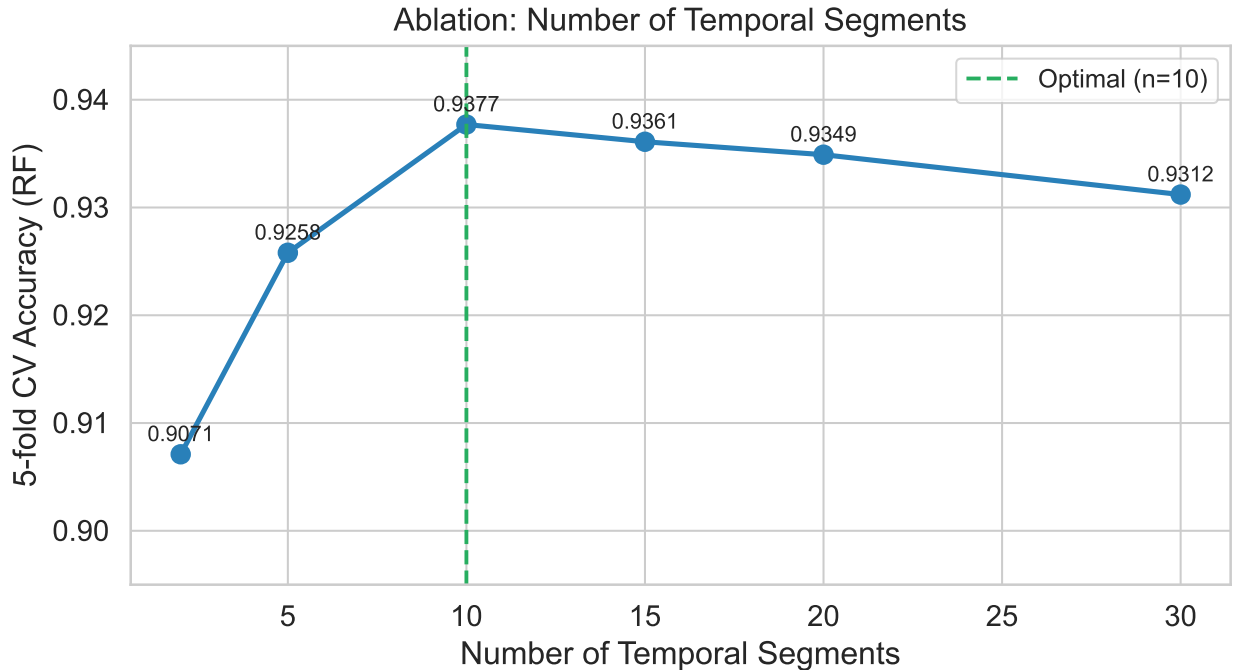


Figure 7: Effect of number of temporal segments on CV accuracy. I found the optimal at $n=10$ (30-second segments). Fewer segments miss temporal evolution; more segments make each window too short for stable statistics.

I found the optimum at **n=10** (30-second windows). With n=2 the model loses most temporal information; with n=30 each window contains only ~10 samples, which is too few for stable mean and std estimates. The performance is relatively flat between n=10 and n=20, so I chose n=10 as it keeps the feature count manageable.

4.4 Outlier Clipping Threshold

I tested different clipping thresholds to assess sensitivity:

Threshold	CV Accuracy
No clipping	93.5%
± 5	93.7%
± 10	93.8%
± 15	93.7%
± 20	93.5%

The result is robust: performance changes by at most 0.3pp across the range. I chose ± 10 as the default, which cleanly removes implausible sensor spikes without affecting the bulk of the distribution.

5 Final Model & Kaggle Results

5.1 Model Architecture

I submitted an **ensemble of gradient boosted trees and a Random Forest** trained on 716 hand-crafted features.

Feature set (716 total):

Group	Features	Description
mean_x/y/z stats+diff	78	18 stats + 8 trend features per mean channel
std_x/y/z stats+diff	78	Same for each std channel
acc_mag + std_mag stats+diff	52	Vector magnitude channels
Cross-axis magnitudes stats+diff	156	xy_mag, xz_mag, yz_mag and std variants
Sum features stats+diff	52	mean_sum = mean_x+y+z; std_sum
FFT + peak features	50	5 channels $\times$ (6 FFT + 4 peak features)
10-window statistics	250	5 channels $\times$ 10 segments $\times$ 5 stats

Training setup:

- **Cross-validation:** I used GroupKFold(5) on user identity — no user appears in both train and validation within a fold, which mirrors the competition’s train/test structure.
- **Classifier:** LightGBM with `num_leaves=31`, `learning_rate=0.01`, `n_estimators=5000` with early stopping (`patience=300`). The lower learning rate (0.01 vs the 0.03 used in earlier runs) was a key final change: it produced best iterations of [587, 507, 468, 378, 477] (avg 483), roughly 3× the ~160 achieved at `lr=0.03`, and resulted in the best Kaggle score of 0.7906.
- **Class imbalance:** I used `sample_weight='balanced'` so that C2 (3.2%) and C4 (1.3%) receive proportionally higher weight during training.
- **Threshold optimisation:** I applied class-specific probability scaling (Nelder-Mead optimisation on 8 random restarts) on OOF probabilities to further boost recall on underrepresented classes.
- **Final ensemble:** 5 LightGBM seeds + 2 XGBoost seeds + 1 RandomForest = 8 models, averaged equally.

5.2 Performance Progression

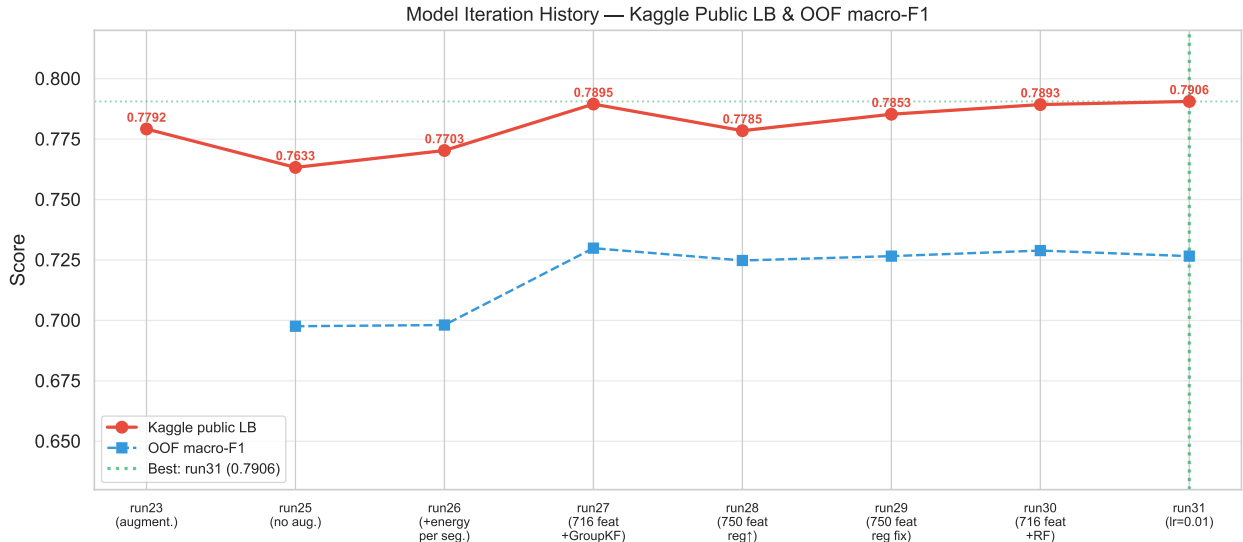


Figure 8: Kaggle public LB score and OOF macro-F1 across my model iterations. The biggest jump is at run27 (axis-specific mean features + GroupKFold). run31 achieves the best Kaggle score (0.7906) with `lr=0.01` despite a slightly lower OOF than run30, showing the lower learning rate generalises better to unseen users.

5.3 Confusion Matrix Analysis

Per-class OOF performance (run31):

Class	Precision	Recall	F1
C0	0.961	0.970	0.965
C1	0.899	0.900	0.899
C2	0.298	0.204	0.242

Class	Precision	Recall	F1
C3	0.625	0.761	0.686
C4	0.893	0.880	0.887
C5	0.736	0.631	0.680
Macro avg	0.735	0.724	0.727

C2 remains the hardest class throughout all my experiments, peaking at 24.2% F1. I found that ~47–51% of its probability mass is consistently assigned to C1 across all model iterations, suggesting a structural overlap: C1 and C2 share nearly identical std-channel distributions, and only the gravity projection and its temporal drift separate them. Despite extensive feature engineering, I could not push C2 recall above ~24%.

An interesting finding emerged from the final run: **run31 achieves the best Kaggle score (0.7906) despite a lower OOF than run30 (0.7266 vs 0.7289)**, yielding an increased OOF→Kaggle gap:

Run	OOF macro-F1	Kaggle public LB	Gap
run27	~0.730	0.7895	+0.060
run29	0.7266	0.7853	+0.059
run30	0.7289	0.7893	+0.060
run31	0.7266	0.7906	+0.064

I interpret this as a bias-variance trade-off: with lr=0.01 and ~483 best iterations (vs ~161 at lr=0.03), the model takes smaller, more conservative steps. The OOF score is slightly lower because the model under-fits relative to training users — but on entirely unseen test users, this conservative optimum generalises better, as it is less likely to have captured training-user-specific signal patterns.

6 Summary

Aspect	Decision	Rationale
Normalisation	None	Gravity projection in mean_x/y/z is class-discriminative
CV strategy	GroupKFold by user	Mirrors train/test split; prevents user leakage
Feature set	716 hand-crafted	Axis stats + temporal segments + FFT + windows
Model	LightGBM ensemble	Best CV accuracy; handles imbalance via sample weights
Imbalance handling	Balanced weights + threshold opt.	Macro-F1 objective; C2, C4, C5 severely underrepresented
Best Kaggle score	0.7906 (public LB, 50% of test)	run31 (lr=0.01, patience=300)

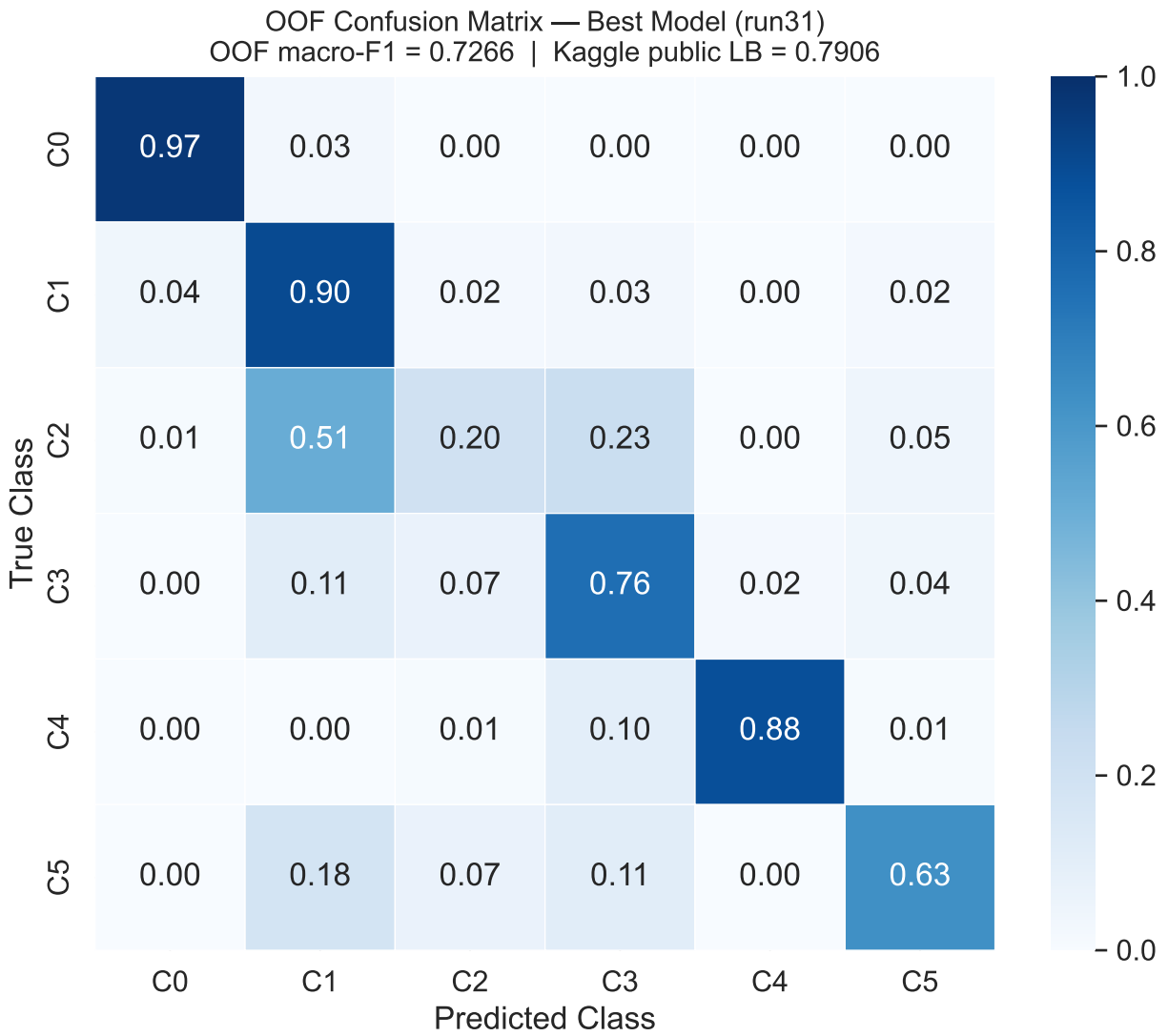


Figure 9: OOF normalised confusion matrix for my best model (run31). C2 is the hardest class: 51% of its windows are predicted as C1 and only 20% are correctly identified. C0 and C4 are classified reliably (97% and 88%).